

StoreLens

Technical Architecture Guide

Insights, observation contracts, exact analytics, plane-aware geometry, and worker lifecycle control

Scope	Source of truth	Document status
Current StoreLens POC implementation	Repository code and local documentation	Implementation reference - 17 July 2026

Central rule

Workers post what a model observed. StoreLens preserves and enriches those rows, then derives analytics, alerts, and insight views. Workers do not post trusted business aggregates.

Prepared from the current working tree, including uncommitted plane-aware geometry and heartbeat-backed worker changes. Roadmap-only capabilities are explicitly separated from implemented behavior.

1. Executive technical summary

StoreLens is a modular FastAPI control plane for computer-vision observations in physical spaces. It stores camera connection metadata, snapshots, a map, named zones, floor and named-plane homographies, per-camera decision regions, jobs, worker instances, raw events, alert rules, and structured insight definitions. It does not contain a general computer-vision runtime and it does not subscribe to every camera continuously.

Layer	Owns	Does not own
External worker	Camera connection, model inference, tracking/classification, raw observation emission	Dashboard cards, derived dwell/state duration, persistent process supervision
REST API	Stable worker contract, batch ingestion, enrichment, storage, analytics, lifecycle protocol	Model choice, arbitrary worker execution
MCP server	Agent discovery, configuration, verification, skill delivery	Persistent camera subscription or production worker runtime
Dashboard	Configuration, inspection, insight catalogue, review, cooperative worker commands	Long-running CV processes

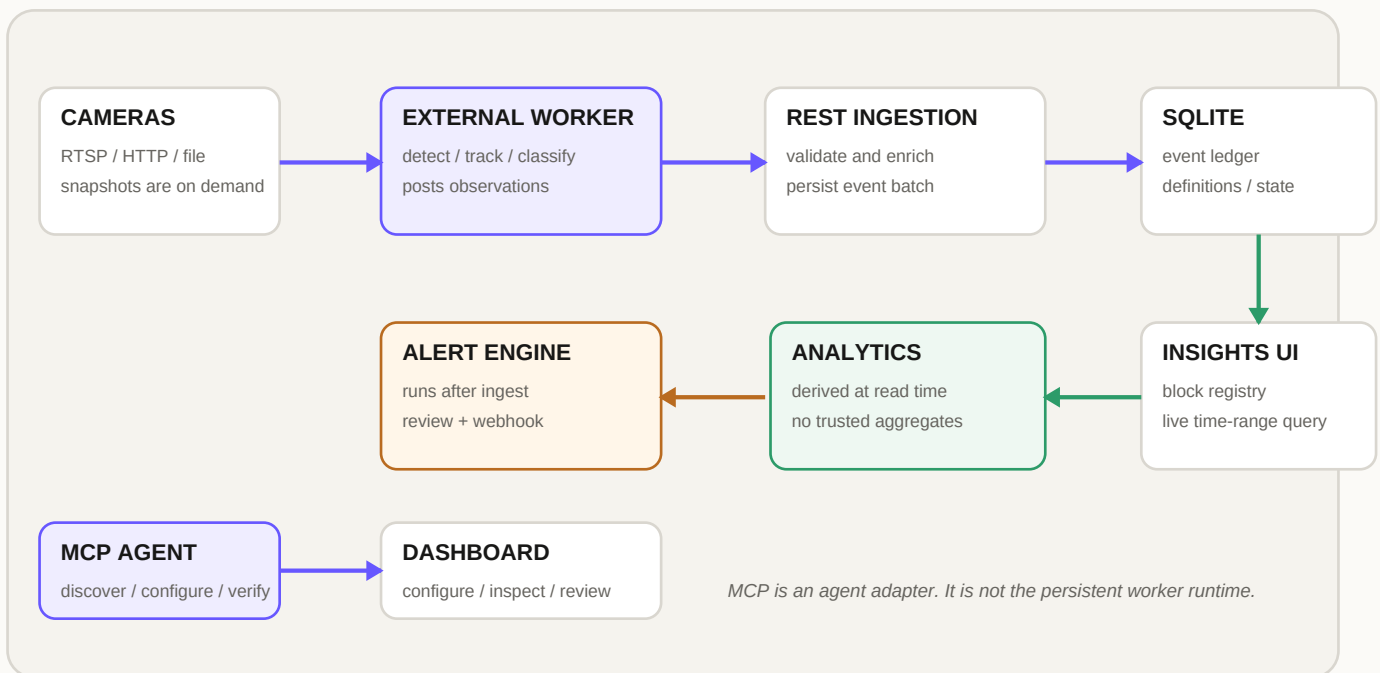


Figure 1. Current control and data paths. Analytics are queried from stored events; an insight stores a definition, not a materialized result.

Implemented invariants

- Dwell is derived from paired `zone_enter` and `zone_exit` events. The deprecated `zone_dwell` value is stored but ignored.
- State duration is derived from consecutive `state_change` timestamps. A worker posts only the new label and a startup anchor.
- A pixel point is projected onto exactly one selected plane: worker-supplied map point, named projection surface, or floor calibration.
- Geometry edits increment revision numbers. Future events use new geometry; historical rows are not silently re-enriched.
- Worker control is cooperative. StoreLens records desired state and reports it in heartbeat responses; a supervisor must perform an actual restart.

Contents

1. Executive technical summary	2
Implemented invariants	2
Contents	3
2. Architecture, persistence, and time	5
2.1 Runtime components	5
2.2 Main persisted records	5
2.3 Two timestamps	5
3. Worker observation contract	6
3.1 Event fields	6
3.2 What each worker should post	6
3.3 Posting examples	7
3.4 Operational posting guidance	7
4. Event ingestion and enrichment	8
4.1 Batch transaction path	8
4.2 Zone assignment precedence	8
4.3 Stored provenance	8
5. Plane-aware geometry	9
5.1 Global zones	9
5.2 Floor calibration and named projection surfaces	9
5.3 Camera zone views	10
5.4 Membership rules and formulas	10
5.5 Projection selection	10
5.6 Bed configuration example	10
5.7 Geometry limitations	11
6. Analytics datasets and exact calculations	11
6.1 Summary	11
6.2 Heatmap	11
6.3 Occupancy / presence over time	11
6.4 Labelled counts over time	12
6.5 Dwell	12
6.6 Transitions / zone-to-zone flow	12
6.7 State timeline	13
6.8 Dataset parameter reference	13
7. How insights work	13
7.1 Definition, not result	13
7.2 Valid block/dataset combinations	14

7.3 Creation paths	14
7.4 Data-aware templates	14
7.5 Insight lifecycle and catalogue behavior	14
8. Worker lifecycle and update/control system	15
8.1 Job versus worker instance	15
8.2 Registration	15
8.3 Heartbeats	15
8.4 Stop and restart commands	16
8.5 Job state interactions	16
8.6 What the current update system does not yet provide	16
9. Alerts, live updates, and review	16
9.1 Alert calculations	16
9.2 Webhooks and review	17
9.3 Server-Sent Events	17
10. Security, deployment, and technical limits	17
10.1 Current protection model	17
10.2 Accuracy and meaning boundaries	17
10.3 Current POC architecture limits	17
10.4 Roadmap-only concepts	18
11. End-to-end technical runbook	18
11.1 Configure	18
11.2 Build and run a bounded worker	18
11.3 Verify observations	18
11.4 Verify analytics before publishing	18
11.5 Bed count insight definition	19
12. API and MCP reference map	19
12.1 REST endpoint groups	19
12.2 MCP role	19
12.3 Python SDK behavior	19
13. Implementation source map	20

2. Architecture, persistence, and time

2.1 Runtime components

Component	Implementation	Important behavior
Web application	FastAPI + static React/Vite build	All REST routes use /api/v1; OpenAPI is exposed at /docs.
Database	SQLite with WAL mode	Each helper opens a new connection. Event batches use one executemany transaction; related job and alert writes are separate transactions.
Live stream	In-memory Server-Sent Events broker	First 25 enriched events in a batch are broadcast; larger batches also emit one summary. Slow clients can lose live messages without affecting persistence.
Snapshots	On-demand OpenCV capture	Status is the last snapshot result, not continuous stream health. WebRTC capture is delegated to workers.
Agent bridge	FastMCP process	Calls REST and returns snapshots/tools/skills. It must run for an MCP client, but workers do not depend on it.

2.2 Main persisted records

Table	Purpose	Key relationships
stores	Workspace identity, dimensions, map document, data state, space type	The POC initializes workspace ID 1.
sources	Camera protocol, credentials, snapshot status, map placement, floor calibration	Calibration JSON contains point pairs, H, error, frame size, and revision.
zones	Canonical named polygons in map metres	A zone is geometry + semantics only; it creates no alert or worker behavior.
projection_surfaces	Source-specific plane homographies	Named mattress/table/shelf/conveyor/platform/custom planes.
zone_views	Per-zone, per-source visible polygon and decision ROI	Optional projection surface and one membership rule.
jobs	Analysis registration and counters	Describes intended analysis, not process liveness.
worker_instances	Concrete process registrations, heartbeats, metrics, desired state	Latest worker is attached to a job response.
events	Evidence-rich observation ledger	Stores observation time, ingestion time, geometry result, revisions, and attributes.
insight_definitions	Saved question + block + dataset + params	No arbitrary frontend code or SQL is accepted.
alert_rules / alerts	Ingestion-triggered rules and human review records	Webhook delivery is best-effort.

2.3 Two timestamps

Every event has two important times:

Field	Meaning	Use
t s	Observation time supplied by the worker, or server time if omitted	All time-range analytics, transitions, dwell, state durations, alert event time
created_at	Ingestion time generated when the row is inserted	Operational latency, retry/backfill diagnosis, audit of when the server received the row

Backfills

A worker may post old observation timestamps. Charts place them at t s, while Detections shows both times. Job last_event_at is the maximum observation timestamp in the submitted batch, not ingestion time.

3. Worker observation contract

3.1 Event fields

Field	Expected content	Server behavior
ts	Epoch seconds, ISO timestamp, or omitted	Parsed to epoch seconds; omitted means ingestion time is used as observation time.
source_id	Camera source ID	Required for source-specific calibration, surface, and zone-view enrichment.
event_type	detection, zone_enter, zone_exit, transition, state_change, count, custom	zone_dwell is accepted only for compatibility and ignored by derived analytics.
track_id	Stable anonymous ID within the run	Required for occupancy, dwell, and flow. Namespace it by job/run/source to prevent cross-worker collisions.
point_px	Representative camera point {x,y}	Preferred over bbox-derived bottom centre when the physical meaning is known.
point_map	Map metres {x,y}	Takes precedence over server projection and is recorded as worker_point_map.
bbox	[x,y,w,h] in pixels	Preserved; if point_px is absent, bottom-centre becomes the representative point.
keypoints	List of {x,y,name?,confidence?}	Preserved and usable by a keypoints_inside zone-view rule.
mask	Compressed/RLE object or string	Preserved but not decoded or used for membership in the current server.
point_kind	feet, hip_center, torso_center, mask_centroid, custom	Documents what the point physically represents; no automatic validation.
projection_surface_id	Named plane ID	Selects the non-floor homography when no worker point_map is supplied.
zone_view_id	Explicit camera ROI provenance	Forces that view after source validation and can assign its zone.
zone_id / zone	Explicit ID or case-insensitive name	Highest-priority zone assignment. Best for aggregate count samples.
value	Numeric raw sample	Used by count events. It is not a place for derived dwell or duration.
label	Count class or state name	Groups count curves and identifies state changes.
attributes	Free JSON dictionary	Model/version/confidence/domain metadata; selected keys can group dwell.

3.2 What each worker should post

Question	Post	Do not post	Platform result
Where are people/objects?	detection rows with position evidence and optional stable track_id	A precomputed heatmap	Heatmap, occupancy, map assignment
How many are visible over time?	count rows with label + numeric value per sample; explicit zone_id for a zone aggregate	A cumulative total or daily average	Averaged count curve by label
How long in a zone?	zone_enter and zone_exit with the same track_id + zone_id	zone_dwell or a duration field	Completed and open dwell visits
How do tracks move?	zone_enter sequence, or zoned detections as fallback	A precomputed flow matrix	Consecutive zone links
How long was equipment open/on?	state_change on startup and only on label flips	State duration	State segments and totals

3.3 Posting examples

Individual person on an elevated mattress plane

```
{
  "ts": 1784322942.125,
  "source_id": 7,
  "event_type": "detection",
  "track_id": "job42-run3-cam7-t18",
  "bbox": [412, 188, 246, 134],
  "point_px": {"x": 531, "y": 247},
  "point_kind": "hip_center",
  "projection_surface_id": 4,
  "attributes": {
    "class": "person", "confidence": 0.93,
    "model": "pose-model", "model_version": "1.2"
  }
}
```

Bed population sample

```
{
  "ts": 1784322942.125,
  "source_id": 7,
  "event_type": "count",
  "zone_id": 3,
  "label": "people_on_bed",
  "value": 1,
  "attributes": {"model": "pose-model", "model_version": "1.2"}
}
```

The aggregate count has no individual geometry, so the worker should supply the Bed zone explicitly. If the worker posts only individual detections, use the occupancy dataset instead of the counts dataset.

Derived dwell inputs

```
{
  "ts": 1784322900.000, "source_id": 7, "event_type": "zone_enter",
  "track_id": "job42-run3-cam7-t18", "zone_id": 3
}, {
  "ts": 1784322964.400, "source_id": 7, "event_type": "zone_exit",
  "track_id": "job42-run3-cam7-t18", "zone_id": 3
}
```

State timeline inputs

```
{
  "ts": 1784322800.000, "source_id": 9,
  "event_type": "state_change", "label": "closed"
}, {
  "ts": 1784322910.000, "source_id": 9,
  "event_type": "state_change", "label": "open"
}
```

3.4 Operational posting guidance

- Register a job before posting and attach its `job_id` to the whole event batch.
- Register the concrete worker only after the process has actually started.
- Batch at most 5,000 events. A typical continuous worker buffers 100-500 rows and flushes every 1-5 seconds.
- Sample individual detections around 1-2 Hz per active track unless the measurement requires another rate.
- Close open zone visits with `zone_exit` during clean shutdown.
- Record model name/version, confidence, and validation metadata in attributes.
- Use globally collision-resistant track IDs across worker runs because several analytics do not filter by `job_id`.

Retry warning

The current POC has no idempotency key or duplicate-batch protection. Blindly retrying an accepted request can duplicate observations and alter analytics.

4. Event ingestion and enrichment

4.1 Batch transaction path

Stage	Current behavior
1. Validate	Reject empty-invalid types, malformed points/keypoints, non-positive bbox sizes, batches over 5,000, and unknown supplied job IDs.
2. Derive representative point	Use explicit point_px. Otherwise, bbox bottom-centre is $x + w/2$, $y + h$ and point_kind becomes bbox_bottom_center.
3. Match zone view	If zone_view_id is explicit, validate source ownership. Otherwise, when no explicit zone exists, score every view for the source and choose the highest passing score; ties prefer the lower view ID.
4. Select projection	Worker point_map wins. Else use explicit surface, then matched view surface, then source floor calibration.
5. Assign zone	Explicit ID -> explicit name -> matched zone view -> first global map polygon containing point_map.
6. Persist	Insert the complete batch with evidence, method names, revisions, attributes, and ingestion timestamps.
7. Update job	Increment event_count and set last_event_at to the maximum event ts in the batch.
8. Side effects	Evaluate alert rules, publish up to 25 live event messages, publish alert messages, and return enrichment counts.

4.2 Zone assignment precedence

```

explicit zone_id
else explicit zone name
  else passing camera zone view
    else projected map point inside global zone polygon
      else zone_id remains null

```

The automatic server population is therefore conditional. A detection needs a camera source plus usable point/bbox/keypoints for a zone view, or a point that can be mapped into the global zone polygon. A count sample normally has neither individual point nor bbox, so an explicit zone_id is appropriate.

4.3 Stored provenance

Stored field	Example	Why it matters
projection_method	floor, surface:Mattress plane, worker_point_map	Explains how x_map/y_map was obtained.
zone_assignment_method	explicit_zone_id, zone_view:keypoints_inside, map_point	Explains why zone_id was populated.
projection_surface_id / zone_view_id	4 / 12	Links the event to selected geometry records.
zone_revision	3	Records which zone revision number was current at ingestion.
calibration_revision / surface_revision / zone_view_revision	2 / 1 / 4	Records the geometry version counters used.
bbox / keypoints / mask / point_kind	Original model evidence	Allows review of the decision and future reprocessing outside the current API.

Revision limitation

The event stores revision numbers, but the POC does not maintain immutable historical copies of every zone polygon or homography. After an edit, the row proves which revision number was used, but the old geometry definition is not recoverable from a dedicated revision table.

5. Plane-aware geometry

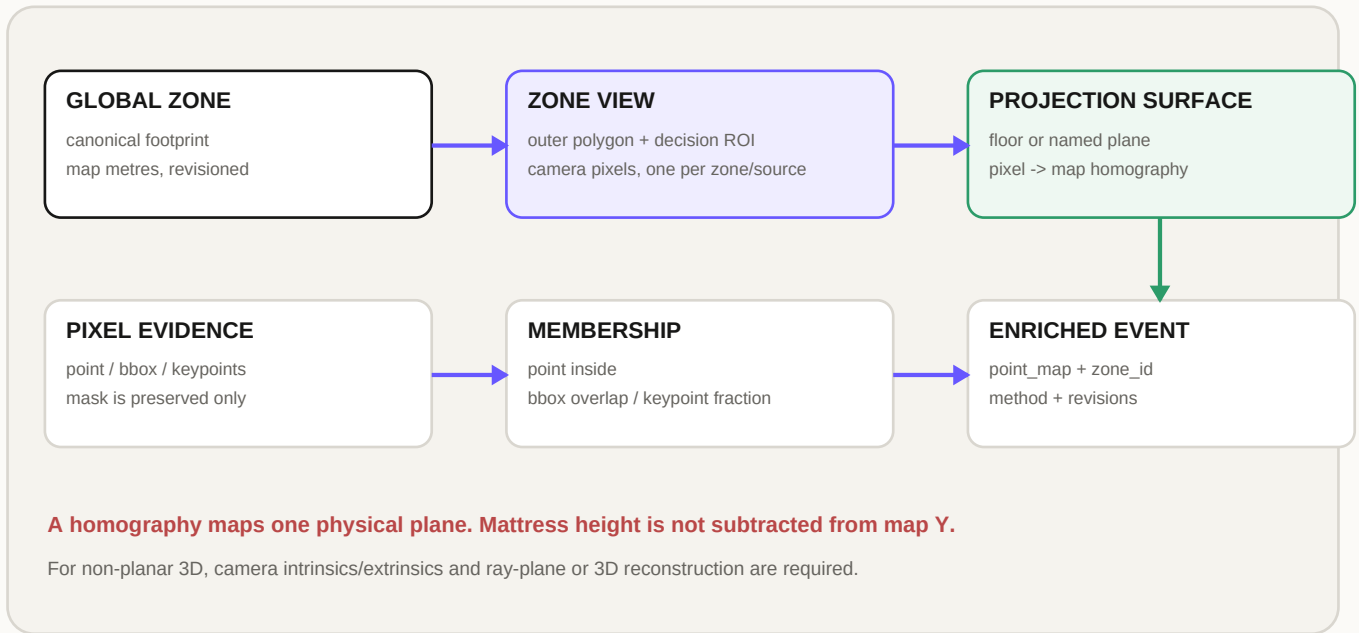


Figure 2. The canonical map zone, camera evidence, and selected physical plane are separate records.

5.1 Global zones

A zone is a named simple polygon in map metres with a semantic type and colour. It is the canonical physical footprint used by map analytics. A zone type such as restricted, queue, hall, or classroom does not create behavior. Alerts and analyses are configured separately.

- At least three map points are required.
- A zone may be created from camera pixels only through the source floor calibration; this path is not appropriate for an elevated surface.
- Updating name, type, colour, or polygon increments the zone revision.
- Deleting a zone deletes its zone views but not historical event rows; without foreign keys, old zone IDs can become orphaned.

5.2 Floor calibration and named projection surfaces

Both floor calibration and named surfaces store a 3×3 homography H from camera pixels to map coordinates on one physical plane. At least four pixel/map correspondences are required. The implementation uses normalized Direct Linear Transform (DLT), solves with singular value decomposition, denormalizes H , and normalizes $H[2,2]$ to 1.

```

For pixel  $p = [u, v, 1]^T$ :
   $q = H p$ 
   $x_{map} = q_x / q_w$ 
   $y_{map} = q_y / q_w$ 

Reported error_m:
  mean over calibration pairs of
   $\sqrt{(x_{projected} - x_{target})^2 + (y_{projected} - y_{target})^2}$ 
  
```

The inverse matrix projects map points back into the camera frame. The Configure UI uses this to project a map footprint through the selected plane.

Plane	How configured	Correct representative point
Floor	Fixed floor landmarks paired with map metres	Feet or bbox bottom-centre for standing/walking subjects
Mattress / table / shelf	Visible surface corners paired with the zone footprint corners	A point physically on that plane, such as hip/torso centre for a lying person

Plane	How configured	Correct representative point
Worker map point	Worker sends point_map directly	Worker assumes responsibility for the coordinate meaning and calibration
Height is metadata The saved height_m is descriptive and is not used in H. The separate correspondence set already defines the elevated plane. Subtracting mattress height from map Y is dimensionally wrong because map Y is horizontal, while height is vertical Z.		

5.3 Camera zone views

A zone view is unique for one (zone, source) pair and stores two camera-pixel polygons:

- **Outer polygon:** the visible extent of the physical zone in the image.
- **Detection polygon:** an inset decision ROI used for membership. If omitted on creation, it defaults to the outer polygon.

The UI can unproject the zone footprint through the chosen floor/surface homography. It then creates a nominal 10% inset by multiplying every vertex displacement from the polygon centroid by 0.9. This is radial scaling, not a constant-distance geometric inset, and should be adjusted visually.

5.4 Membership rules and formulas

Rule	Score	Pass condition	Best use
point	1 when the representative point is inside the decision polygon, else 0	score >= threshold	Compact objects or a reliable semantic point
bbox_overlap	area(decision polygon intersect bbox) / area(bbox)	score >= threshold	Coarse presence when a box substantially overlaps the zone
keypoints_inside	inside positive-confidence keypoints / all positive-confidence keypoints	inside count >= min_keypoints and score >= threshold	Lying, seated, or partially visible people

For automatic matching, every zone view on the source is scored. The highest passing score wins. Equal scores prefer the lower zone-view ID. Explicit zone or view IDs remove ambiguity.

5.5 Projection selection

```

if event.point_map exists:
    use it                # projection_method = worker_point_map
else if event.projection_surface_id exists:
    project point_px with that surface
else if a matched zone view has a surface:
    project point_px with the view surface
else if source floor calibration exists:
    project point_px with floor H
else:
    keep pixel evidence only

```

5.6 Bed configuration example

Step	Configuration	Verification
1	Draw the Bed global footprint correctly in map metres.	Zone revision is visible.
2	Create a Mattress plane from at least four visible mattress corners matched in the same order to the footprint corners.	Review error_m and frame dimensions.
3	Create a zone view using the Mattress plane; project the footprint into frame and adjust outer + decision polygons.	Choose keypoints_inside or bbox_overlap.
4	Post a bounded detection with source_id and bbox/keypoints. Prefer a torso/hip point for mattress localization.	Detection shows surface method, Bed assignment, and geometry revisions.
5	For a zone population curve, post count samples with explicit Bed zone_id.	Counts analytics returns only Bed samples when zone-filtered.

5.7 Geometry limitations

- No lens-distortion model, camera intrinsics, extrinsics, ray-plane intersection, or general 3D reconstruction is stored.
- Frame width/height are metadata. A camera resolution change does not automatically rescale saved pixels.
- Masks are stored but not expanded into polygon membership.
- Overlapping global zones use the first containing zone returned by the database when no zone view or explicit zone resolves the ambiguity.
- A bbox-only event derives bottom-centre for projection even when bbox_overlap assigned an elevated zone. For meaningful elevated map localization, provide a semantic point.

6. Analytics datasets and exact calculations

Unless supplied, every analytics query uses `until = now` and `since = until - 86,400` seconds. Analytics are recomputed from event rows at request time.

6.1 Summary

Output	Calculation
events	COUNT(*) where ts is between since and until
tracks	COUNT(DISTINCT track_id) in the selected range across all event types
active_tracks	COUNT(DISTINCT track_id) with ts >= server now - 300 seconds; independent of the requested range
jobs_active	Jobs whose registration status is active
alerts_unacked	Alerts with acknowledged = 0
sources_online / total	Source snapshot statuses; online is not a live stream heartbeat
zones	Current zone row count

The metric block can display summary fields such as tracks, events, or active_tracks. The Tracked visits template uses tracks, but these are distinct anonymous IDs, not validated unique people.

6.2 Heatmap

```
cell = clamp(requested_cell, 0.05, 2.0) metres
cx = CAST(x_map / cell AS INTEGER)
cy = CAST(y_map / cell AS INTEGER)
weight = COUNT(events in cell)
display point = (cx * cell + cell/2, cy * cell + cell/2)
```

Defaults: event_type=detection and cell=0.25 m. Optional filters are job_id and event_type. Only rows with x_map participate. Results are ordered by weight descending and capped at 50,000 cells.

Interpretation Heatmap weight is observation density, not automatically unique-person density or dwell. Posting frequency directly affects intensity. Use a controlled sample rate.

6.3 Occupancy / presence over time

```
effective_bucket_s = max(requested_bucket_s,
                        (until - since) / 500,
                        10)

for each bucket:
  count = COUNT(DISTINCT track_id)
  where ts in bucket and track_id is not null and zone_id is not null
  and optional zone_id matches
```

Default requested bucket is 600 seconds. Empty buckets are explicitly returned with count 0. All zone-assigned event types participate, so this measures distinct tracks *seen in a bucket*, not a continuously maintained instantaneous occupancy state. A zone_exit row can still make a track present in that bucket.

The line block plots the series. A metric + occupancy definition reports the maximum bucket count as Peak per interval.

6.4 Labelled counts over time

```
effective_bucket_s = max(requested_bucket_s,
                        (until - since) / 500,
                        10)

select event_type = count and value is not null
group by bucket and COALESCE(non-empty label, "objects")

point.count = AVG(value)
point.min   = MIN(value)
point.max   = MAX(value)
point.samples = COUNT(*)
```

Default requested bucket is 300 seconds. Optional filters are `zone_id` and `job_id`. Unlike occupancy, missing buckets are not zero-filled. An insight line selects the configured label; if it is missing, the dashboard currently falls back to the first returned series.

Multi-zone caution

Without `zone_id`, samples with the same label from multiple zones are averaged together, not summed. Use a zone filter or distinct labels such as `people_on_bed` and `children_in_hall`.

6.5 Dwell

The derivation reads only `zone_enter` and `zone_exit` rows with both `track_id` and `zone_id`. It looks back `max_dwell_s` before the requested start so a visit that started earlier can close inside the window. Default `max_dwell_s` is 3,600 seconds.

```
key = (track_id, zone_id)

on zone_enter:
  keep the first unmatched enter for key

on matching zone_exit:
  if exit_ts > enter_ts and exit_ts >= since:
    duration = min(exit_ts - enter_ts, max_dwell_s)
    completed = true

after all rows, for every unmatched enter before until:
  duration = min(until - enter_ts, max_dwell_s)
  completed = false
```

Duplicate enters are debounced by keeping the first. Unmatched exits are ignored. Completed visits are attributed to the window containing their exit. Open visits are included up to until and counted in `open_visits`.

Aggregation key is `(zone_id, group)`. Group is the string value of the enter/exit attributes key named by `group_by`, or all. For each group:

```
visits  = number of derived visits
total_s = sum(duration)
avg_s   = total_s / visits
```

A dwell metric across multiple rows computes a visit-weighted average in the dashboard: $\text{sum}(\text{row.avg_s} * \text{row.visits}) / \text{sum}(\text{row.visits})$. Bar blocks show `avg_s`; table blocks show visits, average, and total.

6.6 Transitions / zone-to-zone flow

The server first reads `zone_enter` rows ordered by `track_id` and `ts`. If there are no `zone_enter` rows at all in the range, it falls back to zoned detection rows. For each track, a link is counted when the zone changes and the timestamp difference from the stored previous-zone timestamp is at most `max_gap_s` (default 1,800 seconds). Results are sorted by count descending.

```
if same track and zone != previous_zone and ts - previous_ts <= max_gap_s:
  links[(previous_zone, zone)] += 1

if new track or changed zone:
  previous_track, previous_zone, previous_ts = current row
```

Fallback nuance

With dense detection fallback, previous_ts is set when the current zone is first observed and is not refreshed by repeated detections in that same zone. The 30-minute test can therefore behave like a maximum zone-segment duration, not purely an inactivity gap.

6.7 State timeline

The query loads state_change rows up to until, optionally for one source, and orders them by time. For each change, the segment starts at max(change.ts, since) and ends at the next change timestamp or until. Segments ending before since or lacking a label are skipped. Seconds are summed by label for each source.

```
segment_i.start = max(change_i.ts, since)
segment_i.end   = change_(i+1).ts, or until for the final state
segment_i.seconds = end - start
total[label] = sum(segment.seconds for that label)
```

A state before the selected range acts as the anchor for the range. The last known state is assumed to continue until the next change or until, even through long data gaps.

6.8 Dataset parameter reference

Dataset	Parameters accepted by endpoint	Default / constraint
summary	since, until	24-hour default range
heatmap	since, until, event_type, job_id, cell	detection; 0.25 m; cell clamped 0.05-2.0
occupancy	since, until, bucket_s, zone_id	600 s requested; effective minimum 10 s and <=500 buckets
counts	since, until, bucket_s, zone_id, job_id	300 s requested; effective minimum 10 s and <=500 buckets
dwell	since, until, group_by, zone_id, max_dwell_s	max visit 3,600 s
transitions	since, until, max_gap_s	1,800 s
states	since, until, source_id	last label carries forward

7. How insights work

7.1 Definition, not result

An insight row stores a title, question, visualization block, analytics dataset, query parameters, unit, limitations, pin/order/visibility metadata, creator, and lifecycle status. Opening the Insights page calculates a new since/until range ending at the browser's current time and calls the selected analytics endpoint.

```
insight definition
-> dashboard builds /analytics/{dataset}?since=...&until=...&params...
-> server derives data from events
-> fixed React block renders response
-> unit and limitations are shown on the card
```

The dashboard forwards only a whitelist of parameters: zone_id, label, group_by, source_id, bucket_s, event_type, job_id, cell, and max_dwell_s. Unknown stored params do not become arbitrary queries. Note that max_gap_s is accepted by the transitions API but is not currently on the dashboard whitelist.

7.2 Valid block/dataset combinations

Block	Accepted datasets	Rendered calculation
metric	summary, dwell, occupancy	Selected summary field; weighted dwell average; or peak occupancy bucket
line	occupancy, counts	Presence series or selected count-label series
bar	dwell	Average dwell per zone/group, with visit count detail
table	dwell, transitions	Dwell rows or from/to/count rows
heatmap_map	heatmap	Weighted cells over the floor map
flow_matrix	transitions	Zone-to-zone movement table/matrix
state_timeline	states	Per-source state segments and total seconds by label

The server validates block/dataset compatibility, visibility, creator, and status. It does not currently validate a dataset-specific parameter schema; invalid or unknown parameters can be stored and then ignored or produce an endpoint error.

7.3 Creation paths

Path	Workflow	Strength / limitation
Dashboard template	Add insight -> choose data-aware template -> edit title/question/unit/limitations/pin -> register	Safe and simple, but the current form exposes a Zone selector only for dwell.
MCP agent	list templates -> list existing definitions -> register_insight with block/dataset/params	Can specify zone_id, label, job_id, bucket_s, group_by, source_id, and other supported params.
REST	POST /api/v1/insights	Full structured contract for trusted clients; still limited to the block registry.

7.4 Data-aware templates

Template	Availability test	Generated params
Presence over time	Always shown	occupancy, no zone filter by default
Counts over time	Disabled placeholder until a labelled count exists	One Population - {label} template per distinct count label
Activity heatmap	Any event has x_map	heatmap
Dwell by zone	Any zone_enter exists	dwell
Zone-to-zone flow	Any zone_enter exists	transitions
Tracked visits	Always shown	summary field=tracks
Queue presence - zone	Zone type checkout or queue	metric + dwell + zone_id
State timeline - source	A state_change exists for that source	states + source_id
Dwell by attribute	Attribute key appears in the latest 1,000 zone_enter rows with attributes	dwell + group_by

Template discovery scans the whole current database, not the selected dashboard time range. A template can therefore be available even when the chosen time range has no matching rows.

7.5 Insight lifecycle and catalogue behavior

- Statuses: draft, collecting, validating, ready, degraded, retired.
- The dashboard shows a badge for non-ready states. Retired definitions are filtered from the normal Insights page.

- Visibility can be visible or hidden. Normal list calls exclude hidden definitions.
- Pinned definitions also render on Overview. Pinning does not copy or snapshot data.
- Deleting a definition deletes the card only; underlying events remain.
- Agent-created definitions carry created_by=agent and show an agent badge.

No arbitrary UI

An API or agent can register only supported blocks and datasets. It cannot post React, HTML, JavaScript, SQL, or a custom analytics function as an insight.

8. Worker lifecycle and update/control system

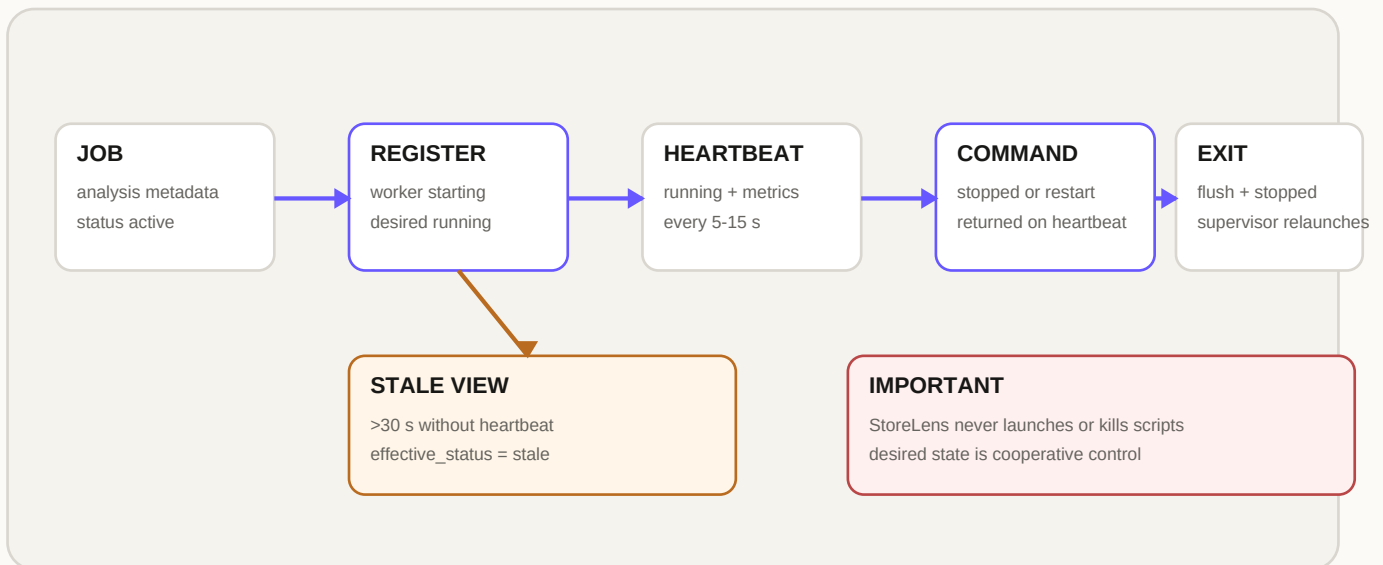


Figure 3. Job registration and worker process liveness are deliberately separate.

8.1 Job versus worker instance

Record	Meaning	Status model
Job	Analysis definition: name, sources, expected event types, event counters	active, paused, done
Worker instance	One concrete running process with external worker_id, version, config, metrics, timestamps	starting, running, stopping, stopped, error; plus computed stale
Desired state	Control request stored on the worker row	running, stopped, restart

A job marked active is not proof that code is running. Runtime truth comes from the latest worker registration and heartbeat. A camera marked online is also separate: it only reflects the last snapshot test.

8.2 Registration

- POST /jobs creates analysis metadata. Source IDs and event types are descriptive lists and are not validated against every later event.
- POST /workers requires an existing job. A missing external worker_id is replaced with a UUID.
- Re-registering the same external worker_id on the same job resets status to starting, desired state to running, clears errors/stopped time, and refreshes start/heartbeat timestamps.
- The same external worker_id cannot move to another job; that returns conflict.

8.3 Heartbeats

A heartbeat replaces the stored metrics dictionary, records status and last_error, updates last_heartbeat_at, and sets stopped_at for stopped or error. Valid heartbeat states are starting, running, stopping, stopped, and error.

```

response.should_stop = desired_state in {"stopped", "restart"}
response.restart_requested = desired_state == "restart"

effective_status = "stale" when:
  stored status is starting/running/stopping
  and now - last_heartbeat_at > 30 seconds

```

Workers should heartbeat every 5-15 seconds. Useful metrics include FPS, frames processed, inference latency, queue depth, reconnect count, dropped frames, current state, open visits, and last source-frame time.

8.4 Stop and restart commands

The dashboard and MCP update `desired_state`. The next heartbeat returns the command. The worker must flush, close open visits when appropriate, release camera/model resources, send a terminal stopped/error heartbeat, and exit.

```

while running:
  command = sl.heartbeat(status="running", metrics=metrics)
  if command["should_stop"]:
    close_open_zone_visits()
    sl.flush()
    sl.stop_worker()
    break

# If command["restart_requested"] was true, an external supervisor
# must start the process again. Re-registration resets desired_state."

```

No embedded supervisor

StoreLens does not execute worker paths, spawn arbitrary scripts, kill processes, install model updates, or relaunch after restart. Use systemd, Docker, Kubernetes, Windows Services, or another deployment supervisor.

8.5 Job state interactions

- Pausing or completing a job sets `desired_state=stopped` on its non-terminal worker instances.
- Reactivating a job does not automatically reset an old worker desired state or launch a process. Request restart and rely on a supervisor.
- Deleting a job removes worker rows. Events remain unless `purge_events=true`.
- The dashboard's latest worker is selected by worker creation time, not by best health.

8.6 What the current update system does not yet provide

Missing capability	Current consequence	Roadmap direction
Package/update manifest	No signed model or worker version rollout	Versioned worker packages, staged update, rollback
Checkpoint/cursor	A restarted worker must implement its own resume behavior	Persist processing cursor and commit after accepted observations
Offline buffer	Network interruption handling is worker-specific	Edge agent with durable local queue and gap reporting
Idempotent ingestion	Retries may duplicate rows	Stable observation/batch idempotency keys
Central logs	Only <code>last_error</code> and free-form metrics are stored	Structured logs, health history, diagnostics
True process supervisor	Restart is a request only	Edge runtime supervises start/stop/restart/update

9. Alerts, live updates, and review

9.1 Alert calculations

Enabled rules are evaluated after each event batch is persisted, so derivation queries can see rows from that batch. A rule can fire at most once per evaluation and is throttled by its `last_fired_at` and `cooldown`.

Rule	Calculation	Important semantics
dwelling_exceeds	On a new zone_exit, derive time since the latest matching enter. Also scan unmatched open visits at batch max ts.	Derived only; zone_dwelling values are ignored. Ongoing conditions require new batches to trigger evaluation.
occupancy_exceeds	COUNT(DISTINCT track_id) in [batch_max_ts-window_s, batch_max_ts], optionally by zone	Counts any event type with track + zone in the window.
state_alert	Immediate label match, or derived duration from previous state when it ends; also tests ongoing state	Last state carries forward. Duration is not worker-supplied.
event_match	Match event_type, optional zone, optional attribute key/value	String comparison is used for attribute values.

9.2 Webhooks and review

- An alert row is persisted before webhook delivery.
- Webhook POST runs on a daemon thread with a five-second timeout. Failures are swallowed; there is no retry queue or delivery audit.
- Review statuses are new, in_review, resolved, and dismissed. Notes and resolved timestamps are stored.

9.3 Server-Sent Events

- The endpoint sends a retry hint of 3,000 ms and a keepalive comment every 15 seconds.
- Each client queue holds up to 1,000 messages. Messages are dropped for slow clients rather than blocking ingestion.
- Live messages are an acceleration path only. Durable truth remains in SQLite; charts re-query analytics.

10. Security, deployment, and technical limits

10.1 Current protection model

- If STORELENS_API_KEY is set, every /api request except health requires the matching X-API-Key header or api_key query parameter.
- The same shared key is used by dashboard, workers, and MCP. There are no users, roles, tenant boundaries, or per-worker service identities.
- CORS currently allows all origins, methods, and headers.
- Source credentials are stored in SQLite and can be returned by the secrets=true source endpoint. They are not encrypted by this POC.

10.2 Accuracy and meaning boundaries

Claim	What the implementation actually supports
Unique visitors	Distinct track IDs in a range. Re-identification or tracker resets can count a person again; ID collisions can undercount.
Occupancy	Distinct tracks observed in a bucket, not a maintained enter-minus-exit state.
Wait time	Dwell inside a configured zone; it is not automatically validated queue wait.
Heatmap	Observation frequency by grid cell; sensitive to sampling rate and occlusion.
State duration	Time between labels, assuming the last label remains true through data gaps.
Bed presence	A model estimate governed by selected ROI/plane/rule; not identity or a safety-critical fact.

10.3 Current POC architecture limits

- Single SQLite database and one default workspace; no tenancy isolation or PostgreSQL migrations.
- No foreign-key enforcement, immutable geometry history, event idempotency, processing watermark, replay state projection, or retention jobs.

- Several datasets combine jobs. Occupancy, dwell, transitions, summary tracks, and states are not job-filterable in their current endpoints.
- No durable cross-camera entity model. Track IDs are local temporary identifiers; persistent person identity is explicitly outside the intended initial product.
- No continuous source health, edge-agent health, worker logs, or end-to-end latency monitor.
- No strong webhook delivery guarantees.
- No arbitrary custom insight dataset; new computations require server and renderer changes.

10.4 Roadmap-only concepts

The repository roadmap proposes PostgreSQL, tenancy, idempotent observations, track segments, entity/catalogue knowledge for products, current-state projections, reconciliations, checkpoints, secure edge enrollment, offline buffering, signed packages, staged rollout, rollback, validation gates, retention, audit, and operational monitoring. These are design directions, not current implementation guarantees.

Medicine warehouse example

The POC can store labelled observations and curves, but it is not a regulated inventory or dispensing system. A durable medicine state would need catalogue identity, checkpoints, freshness, uncertainty, reconciliation, audit, and independent controls.

11. End-to-end technical runbook

11.1 Configure

- Create/test a source and confirm a real snapshot.
- Draw the global map and physical zone footprints.
- Calibrate the floor with at least four stable points.
- For an elevated target, create a named plane and a camera zone view with an adjusted decision ROI.

11.2 Build and run a bounded worker

- Discover source, zones, surfaces, and views from REST/MCP; do not hardcode assumed IDs from a demo.
- Define the measurement: individual detections, per-frame count, enter/exit tracking, or state flips.
- Register the job, then start the process, then register the worker instance.
- Heartbeat every 5-15 seconds, buffer events, obey `should_stop`, and flush cleanly.

11.3 Verify observations

Check	Pass condition
Timestamps	Observation ts matches capture time; <code>created_at</code> shows reasonable ingestion delay.
Attribution	Correct <code>job_id</code> , <code>source_id</code> , <code>track/label</code> , model metadata.
Geometry	Correct <code>point_kind</code> , selected surface, zone assignment method, and revisions.
Track stability	IDs remain stable long enough for the intended occupancy/dwell/flow calculation.
Sampling	No accidental per-frame flood that biases heatmap or storage.
Shutdown	Open visits are closed and worker sends <code>stopped/error</code> .

11.4 Verify analytics before publishing

- Query the exact dataset with the intended since/until, zone, label, job, source, and bucket parameters.
- Compare a small time window against Detections manually.

- Check false positives, missed subjects, geometry edge cases, track resets, and gaps.
- Register one useful insight definition, state limitations honestly, and pin only when requested.

11.5 Bed count insight definition

```
{
  "title": "People on bed over time",
  "question": "How many people are detected on the Bed over time?",
  "block": "line",
  "dataset": "counts",
  "params": {
    "label": "people_on_bed",
    "zone_id": 3,
    "job_id": 42,
    "bucket_s": 10
  },
  "unit": "people",
  "limitations": "Average model-reported count per interval; not identity or a safety signal.",
  "pinned": false
}
```

The Add Insight UI currently cannot expose every parameter above. An agent can register the full definition through MCP, or a trusted client can use REST. The count template appears after at least one labelled count event exists.

12. API and MCP reference map

12.1 REST endpoint groups

Concern	Primary endpoints
Health and docs	GET /api/v1/health; GET /docs
Sources	GET/POST /sources; snapshot; placement; calibration; project; unproject
Zones and geometry	GET/POST/PUT/DELETE /zones; /projection-surfaces; /zone-views
Jobs and workers	GET/POST/PUT/DELETE /jobs; GET/POST /workers; heartbeat; desired-state
Observations	POST /events; GET /events; GET /stream
Analytics	GET /analytics/summary heatmap dwell occupancy counts transitions states
Insights	GET/POST /insights; GET /insights/templates; PUT/DELETE /insights/{id}
Alerts/review	GET/POST /alert-rules; GET/PUT /alerts; acknowledgement endpoints

12.2 MCP role

The MCP server mirrors bounded agent operations: discover sources/map/zones/snapshots, manage geometry, register jobs/workers, submit and inspect events, query analytics, register insights, create alerts, and load StoreLens skills. It obtains authoritative shapes by calling REST. Persistent workers should call REST through the Python SDK and should not require an MCP client to stay connected.

12.3 Python SDK behavior

- Uses requests.Session, optional X-API-Key, and a 30-second request timeout.
- Buffers events in memory and auto-flushes at batch_size; atexit attempts a final flush.
- Provides discovery, job/worker registration, heartbeats, project/unproject, capture opening, and a simple greedy centroid tracker.
- The centroid tracker greedily associates nearest centroids within max_distance and deletes tracks after max_missed frames. It is a lightweight example, not a production tracker.
- The SDK does not currently retry, persist its buffer, deduplicate, checkpoint video position, or supervise itself.

13. Implementation source map

The document was derived from these repository sources. The code is authoritative where documentation and roadmap language differ.

Area	Primary files
Platform contract	AGENTS.md; README.md; TUTORIAL.md; skills/storelens-platform/SKILL.md; skills/insights/SKILL.md
Application and storage	server/app.py; server/db.py
Event contract and enrichment	server/routers/events.py; dashboard/src/detections.jsx
Analytics and derivation	server/routers/analytics.py; server/services/derive.py
Insights	server/routers/insights.py; dashboard/src/insights.jsx
Geometry	server/routers/geometry.py; server/routers/sources.py; server/routers/zones.py; server/services/homography.py; dashboard/src/camera-geometry.jsx
Workers	server/routers/jobs.py; sdk/python/storelens.py; dashboard/src/pages.jsx
Alerts and live stream	server/services/alert_engine.py; server/routers/alerts.py; server/services/sse.py
Examples	examples/heatmap_tracker.py; examples/dwell_zones.py; examples/fridge_state.py; examples/simulate_children_counts.py; examples/simulate_shoppers.py
Future architecture	PLATFORM_ROADMAP.md (roadmap only; not current guarantees)

Final operating rule

First verify the raw evidence and its geometry in Detections. Then verify the analytics endpoint. Only after both are credible should an insight be registered or an alert be enabled.

End of technical architecture guide.